

SOP 11 — Ingest Match

Camada A.N.I.: Instruments-Tools (ingest_match.py / ping_football_api.py) · Versão: 1.0 (Protocol 0, 2026-09-09)

Golden Rule: mudou a lógica → atualiza o SOP → só então o código.

Objetivo

Obter do provedor de futebol (API-Futebol ou API-Football — decisão humana) o próximo jogo e o estado dos jogos do Flamengo (`profissional_m` como default), normalizar tudo para o contrato `RawInput.Match` e gravar em `.tmp/` + aba cloud `matches`. **Nunca inventar escalação, placar ou jogo.**

Quando roda

- Cron daily ingest 06:30 (próximo jogo / agenda do dia).
- Cron refresh 12:00 e 18:00 (atualização de status/placar).
- Matchday: poll a cada 30s de kickoff–90min até FT+30 (ver SOP 31).
- Phase L: `tools/ping_football_api.py` — GET do próximo jogo (pode ficar mockado se não houver key).

Input

- Credencial `API_FUTEBOL_KEY` em `.env` (provedor escolhido pelo humano).
- Identidade do time: Flamengo (id do time no provedor — fixado na Phase L e registrado em apêndice).
- Default de escopo: profissional masculino (`team_unit = profissional_m`); outras modalidades só quando o jogo for explicitamente pedido.
- Fixture de fallback: `fixtures/match_scheduled.json` / `fixtures/match_live_goal.json`.

Output

- `RawInput.Match` → `.tmp/matches/<match_id>.json` (sempre o estado mais recente do mesmo `match_id`; versionar por `updated_at` no nome se necessário).
- Espelho na cloud: aba `matches` (linha por `match_id`).

Passos determinísticos

1. Ler `.env` (nunca imprimir o valor da key em log).
2. Chamar o endpoint “próximos jogos do time” do provedor (ou ler o fixture em modo mock quando `API_FUTEBOL_KEY` estiver vazia — handshake da Phase L).
3. Selecionar o próximo jogo do `profissional_m` por `kickoff_iso` (o mais próximo no futuro). Se houver mais de um (ex.: 2 competições), escolher o mais próximo e registrar os demais no JSON de saída.
4. Normalizar campos para `RawInput.Match` :
 - `match_id` canônico, ex.: `brasileirao-2026-r24-fla-flu`;

- `kickoff_iso` convertido para America/Sao_Paulo com offset explícito (-03:00);
 - `status` mapeado: `scheduled` | `live` | `finished` (mapeamento exato dos status do provedor fixado na Phase L);
 - `score_home` / `score_away` numéricos ou `null`;
 - `lineup_confirmed` booleano (`true` somente quando o provedor informar escalação oficial);
 - `broadcast` como lista (ex.: `["Globo", "Premiere"]`).
5. Gravar/atualizar `.tmp/matches/<match_id>.json` e a linha na aba `matches` (quando a cloud estiver disponível).
 6. Reportar JSON de saída: `{ "match_id": "...", "status": "...", "kickoff_iso": "...", "mock": true|false }`.

Edge cases

- **Sem key / API fora do ar** → operar em modo mock com fixtures e marcar `mock: true` no relatório; registrar em `progress.md` / `findings.md`. Não bloqueia o resto do pipeline.
- Sem jogo futuro (fim de temporada/intervalo) → `agora.match = null` no payload; registrar no relatório.
- Jogo adiado/cancelado → atualizar `status` e avisar o editor via webhook (SOP 40) se já havia matchday previsto.
- Provedor entrega horário sem offset → assumir America/Sao_Paulo e logar a suposição.
- Time não encontrado no provedor → PARAR (não adivinhar id); pedir humano com print do resultado da busca.

Rate limits / limites conhecidos

- Conforme o plano do provedor (`api-futebol.com.br` e `api-sports.io` têm limites por dia/segundo — fixar valores reais na Phase L em apêndice).
- Poll de matchday: a cada 30s (blueprint); nunca mais rápido que isso.

Testes

- Fixtures: `fixtures/match_scheduled.json` (estado `scheduled`) e `fixtures/match_live_goal.json` (mesmo `match_id` em `live`, 1-0) — a tool deve passar pelos dois estados sem rede.
- Phase L: ping real do próximo jogo; comparar o `match_id` gerado com o padrão dos fixtures.